Clase 13: Big Data, Machine Learning & TinyML en Salud

Semana 13 | Corte 3 — Computación Neuronal, Arquitecturas Embebidas y Laboratorio con XIAO ESP32S3 Sense

Prof. Jefferson Sarmiento Rojas, Ph.D (c)

2026-10-14

Table of contents

Resultados de Aprendizaje Esperados (RAE) 2
1. De la Lógica Simbólica a la Aritmética Neuronal 3
1.1 Cambio de Paradigma: Programación Tradicional vs. Machine Learning . . 3
1.2 Evolución Histórica de la Inteligencia Artificial 3
1.3 El Cuello de Botella de la Ingeniería de Características 5
1.4 La Neurona Artificial como Primitiva de Computación 5
2. Ciclo de Vida del Sistema de Machine Learning (ML Systems Lifecycle) 7
3. El Espectro de la Inteligencia Distribuida: Cloud ML vs. Edge ML vs. TinyML . 7
3.1 Tabla Comparativa de Paradigmas Computacionales 8
3.2 ¿Por qué TinyML en Ingeniería Biomédica? 9
4. Práctica de Laboratorio: Sistemas Inteligentes en Informática Médica 9
4.1 Características de la Plataforma XIAO ESP32S3 Sense 9
4.2 Planteamiento del Problema Hospitalario 10
4.3 Ejercicio 1: Configuración del Entorno de Desarrollo (Arduino IDE) 11
4.4 Ejercicio 2: Verificación de Hardware (Blink y Monitor Serial) 11
4.5 Ejercicio 3: Adquisición de Señal Acústica con Micrófono PDM e I2S 12
4.6 Ejercicio 4: Grabación de Audio WAV en MicroSD y Habilitación de OPI PSRAM 13
4.7 Ejercicio 5 (Resolución del Reto): Visión Artificial Embebida con SenseCraft AI Studio 16
4.8 Integración con Arquitectura IoT / MQTT (Solución Integral) 21
Bibliografía y Referencias 22

Resumen Ejecutivo de la Sesión

En esta sesión profundizamos en la convergencia entre **Big Data**, **Machine Learning (ML)** y el paradigma emergente de **TinyML (Machine Learning Embebido)** aplicado al entorno biomédico y clínico.
A partir de los principios de computación neuronal desarrollados por Reddi et al. en el texto de referencia *Machine Learning Systems* (mlsysbook.ai), analizaremos cómo la transición de la programación tradicional a la aritmética neuronal permite procesar patrones fisiológicos complejos. Examinaremos el **ciclo de vida completo de un sistema de ML** y contrastaremos el espectro de computación distribuida (**Cloud ML vs. Edge ML vs. TinyML**).
En el componente práctico, abordaremos el reto de **identificación automática de equipamiento hospitalario** durante rondas de vigilancia e inducción, utilizando la placa de desarrollo de ultra bajo consumo **Seeed Studio XIAO ESP32S3 Sense**, dotada de cámara digital OV2640 y micrófono PDM. Los estudiantes entrenarán y desplegarán un clasificador neuronal mediante **SenseCraft AI Studio**, integrando la telemetría del equipo con el protocolo **MQTT** para despachar enlaces a manuales de operación en tiempo real.

Resultados de Aprendizaje Esperados (RAE)

Al finalizar esta sesión y la práctica de laboratorio, el estudiante estará en capacidad de:

1. **Identificar los principios fundamentales del aprendizaje automático embebido (TinyML)** y su diferenciación frente a la computación en la nube (Cloud ML) y la programación tradicional basada en reglas.
2. **Comprender la neurona artificial como primitiva computacional**, analizando el flujo de operaciones de suma ponderada (MACs), funciones de activación no lineales y complejidad de memoria (Flash vs. SRAM) en arquitecturas feedforward.
3. **Diseñar y entrenar modelos TinyML para la clasificación visual de patrones biomédicos**, evaluando métricas de precisión, latencia de inferencia y consumo de potencia.
4. **Implementar un sistema inteligente embebido sobre la tarjeta XIAO ESP32S3 Sense**, configurando periféricos de audio digital (I2S/PDM), memoria PSRAM y cámara OV2640.
5. **Vincular la inferencia en el borde (Edge AI) con arquitecturas de interoperabilidad IoT/MQTT**, transmitiendo alertas y enlaces a documentación técnica hospitalaria de forma asíncrona.

---

1. De la Lógica Simbólica a la Aritmética Neuronal

1.1 Cambio de Paradigma: Programación Tradicional vs. Machine Learning

Durante décadas, el desarrollo de software biomédico se basó en el paradigma de **programación clásica o deducción simbólica**: un ingeniero o equipo clínico definía un conjunto explícito de **reglas** (condicionales lógicos `IF-THEN`, umbrales hemodinámicos fijos) y el computador procesaba los **datos** de entrada para producir **respuestas**.

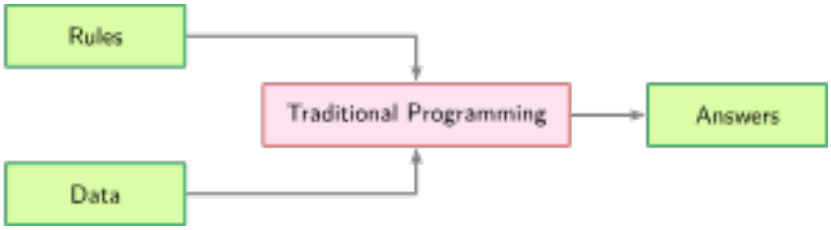


Figure 1: Programación Tradicional: Reglas y Datos producen Respuestas (Fuente: MLSysBook / Reddi et al., 2024)

Sin embargo, en señales biomédicas e imágenes diagnósticas (electrocardiografía, fonocardiografía, retinografía, monitores de signos vitales), la variabilidad biológica, el ruido electrónico y los artefactos de movimiento hacen inviable la formulación manual de reglas universales.

El **Machine Learning (ML)** invierte este flujo mediante un proceso **inductivo**:

$$\text{Datos} + \text{Respuestas (Etiquetas)} \xrightarrow{\text{Entrenamiento}} \text{Reglas (Modelo Aprendido)}$$

El algoritmo aprende de forma estadística una función de aproximación $f(\mathbf{x}) \approx y$ a partir de ejemplos históricos anotados.

1.2 Evolución Histórica de la Inteligencia Artificial

La inteligencia artificial no surgió de forma aislada; es la culminación de varias olas tecnológicas sucesivas:

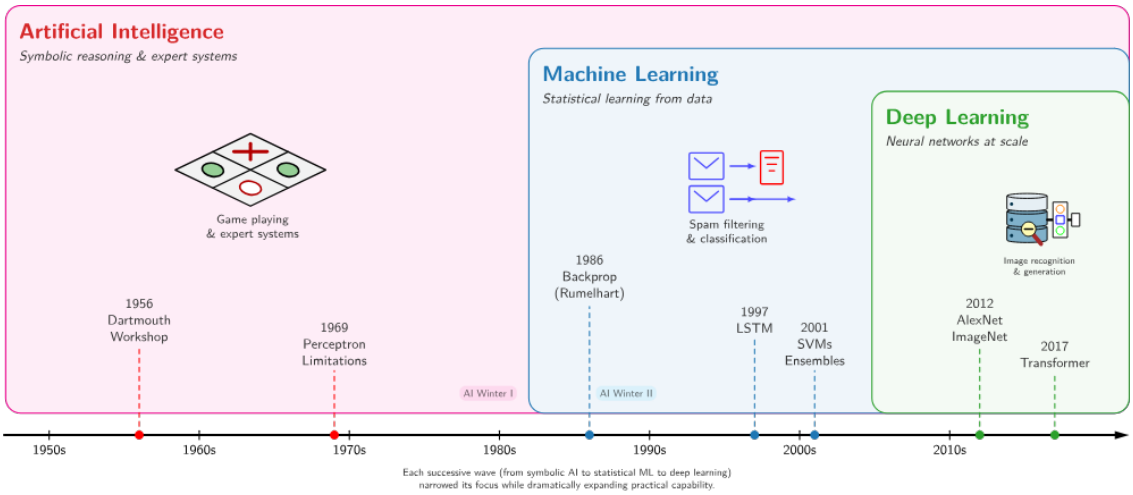


Figure 2: Evolución histórica: De la IA Simbólica al Machine Learning Estadístico y Deep Learning a Escala (Fuente: MLSysBook / Reddi et al., 2024)

Período	Paradigma	Hitos Principales	Características y Limitaciones
1950s – 1970s	**Inteligencia Artificial Simbólica**	• Taller de Dartmouth (1956) • Perceptrón simple de Rosenblatt (1958) • Crítica de Minsky & Papert al problema XOR (1969)	Sistemas expertos basados en lógica proposicional. Fracaso ante problemas no linealmente separables; desencadenó el primer *Invierno de la IA*.
1980s – 2000s	**Machine Learning Estadístico**	• Retropropagación / Backprop (Rumelhart et al., 1986) • Redes Recurrentes LSTM (Hochreiter & Schmidhuber, 1997) • Support Vector Machines (SVM) y Bosques Aleatorios (2001)	Modelado estadístico a partir de datos. Dependencia crítica de la *ingeniería manual de características* (extracción artesanal de picos R en ECG o descriptores de textura).

Período	Paradigma	Hitos Principales	Características y Limitaciones
2010s – Presente	**Deep Learning & Redes a Escala**	• AlexNet y la revolución ImageNet en GPU (2012) • Arquitectura Transformer y Mecanismo de Atención (Vaswani et al., 2017) • Edge AI y TinyML en microcontroladores (Reddi et al., 2020+)	Extracción jerárquica y automatizada de características (Feature Learning). Alto costo computacional que impulsó la aceleración por hardware (TPUs, NPUs) y la cuantización de modelos.

1.3 El Cuello de Botella de la Ingeniería de Características

En el enfoque clásico de Machine Learning, el 80% del tiempo de ingeniería biomédica se invertía en construir transformadas manuales (Transformada Wavelet, Transformada Rápida de Fourier, momentos estadísticos). Si las características extraídas no capturaban la patología, ningún algoritmo lograba clasificarla correctamente.

Las **Redes Neuronales Profundas (Deep Learning)** resuelven este obstáculo al aprender representaciones jerárquicas directamente desde los datos crudos:

- **Capas iniciales:** Detectan bordes, gradientes de intensidad y transiciones de frecuencia.
- **Capas intermedias:** Combinan bordes para formar texturas, formas geométricas o morfologías de ondas clínicas (complejo QRS, onda P).
- **Capas finales:** Representan el objeto completo o estado patológico (taquicardia ventricular, bomba de infusión volumétrica, electrocardiógrafo).

1.4 La Neurona Artificial como Primitiva de Computación

De acuerdo con el modelo formal presentado en *MLSysBook* (Reddi et al., 2024), la computación neuronal se fundamenta en dos pasos matemáticos sucesivos:

```
Entradas (x)    [ Suma Ponderada: z = w·x + b ]    [ Activación No Lineal: a =  (z) ]    Salid
```

1. Suma Ponderada y Operación MAC (Multiply-Accumulate)

Dado un vector de entradas $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$, un vector de pesos $\mathbf{w} = [w_1, w_2, \dots, w_n]^T$ y un escalar de sesgo (*bias*) b:

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

Cada multiplicación $w_i \cdot x_i$ seguida de una suma acumulativa constituye una operación **MAC (Multiply-Accumulate)**, equivalente a **2 FLOPs** (Floating Point Operations). El total de MACs define el presupuesto energético y el tiempo de reloj requerido por el microcontrolador.

2. Funciones de Activación No Lineal

Sin una activación no lineal, una red neuronal de 100 capas colapsaría matemáticamente en una simple transformación lineal afín:

$$f(g(x)) = W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2) = W'x + b'$$

Para romper la linealidad y permitir la aproximación universal de funciones complejas, se aplican funciones de activación:

- **ReLU (Rectified Linear Unit):** $\sigma(z) = \max(0, z)$. Extremadamente eficiente en hardware embebido (un condicional `if (z < 0) return 0;`), base de TinyML.
- **Sigmoide:** $\sigma(z) = \frac{1}{1+e^{-z}}$. Mapea a probabilidades en $[0, 1]$, pero costosa de calcular en procesadores sin unidad de punto flotante exponencial.
- **Softmax:** Utilizada en la capa de salida para problemas de clasificación multiclase (por ejemplo, clasificar entre 4 tipos de equipos médicos):

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

---

2. Ciclo de Vida del Sistema de Machine Learning (ML Systems Lifecycle)

Un modelo de inteligencia artificial en salud no es un archivo aislado; forma parte de una cadena productiva continua compuesta por **10 fases interconectadas** con bucles de retroalimentación activa:

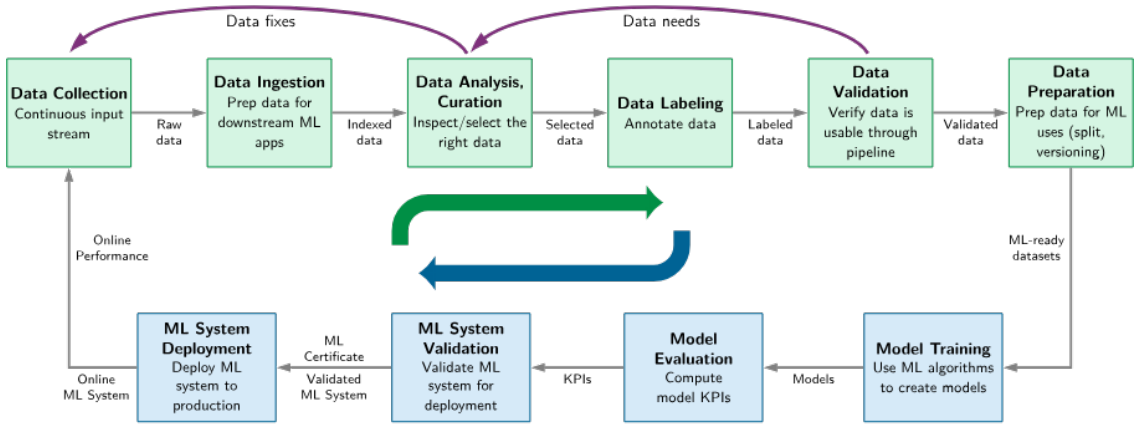


Figure 3: Ciclo de Vida de Sistemas de ML: Desde la adquisición y validación de datos hasta el despliegue e inferencia en producción (Fuente: MLSysBook / Reddi et al., 2024)

Bucles de Retroalimentación en Producción

Como muestra el diagrama de Reddi et al., cuando el sistema está desplegado en un hospital, el monitoreo del rendimiento online retroalimenta continuamente el pipeline: - **Data fixes:** Identifica imágenes mal clasificadas por iluminación o ángulo para corregir el etiquetado. - **Data needs:** Solicita la recolección de nuevas muestras para clases subrepresentadas o nuevos modelos de equipamiento médico ingresados al inventario hospitalario.

---

3. El Espectro de la Inteligencia Distribuida: Cloud ML vs. Edge ML vs. TinyML

La computación inteligente en salud no opera en un único nodo centralizado, sino a lo largo de un espectro continuo de potencia de procesamiento y restricciones energéticas:

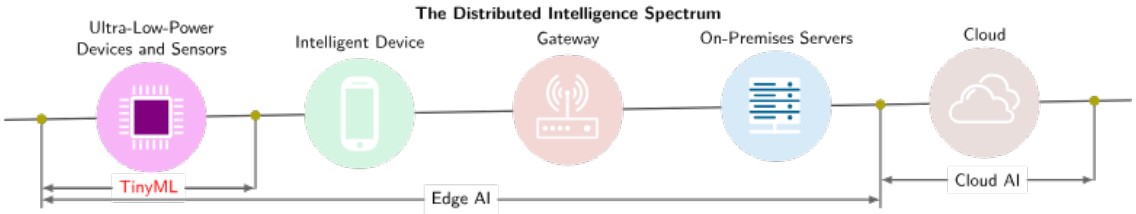


Figure 4: El Espectro de Inteligencia Distribuida: Desde la Nube hasta los Dispositivos de Ultra Bajo Consumo (Fuente: MLSysBook / Reddi et al., 2024)

3.1 Tabla Comparativa de Paradigmas Computacionales

Paradigm	Where	Latency (ms)	Power	Memory	Best For
Cloud ML	Data centers	100-500	MW	TB	Training, complex inference
Edge ML	Local servers	10-100	100 W	GB	Real-time inference, privacy
Mobile ML	Smartphones	5-50	3–5 W	GB	Personal AI, offline
TinyML	Microcontrollers	1-10	mW	KB	Always-on sensing

Figure 5: Parámetros Operacionales de Cloud ML, Edge ML, Mobile ML y TinyML (Fuente: MLSysBook / Reddi et al., 2024)

Paradigma	Ubicación del Cómputo	Latencia Típica	Consumo de Potencia
Cloud ML	Centros de datos remotos (AWS, Azure, GCP)	100 – 500 ms	Megavatios (MW)
Edge ML	Servidores locales hospitalarios / Gateways	10 – 100 ms	~100 W
Mobile ML	Smartphones clínicos, tablets asistenciales	5 – 50 ms	3 – 5 W
TinyML	Microcontroladores embebidos (MCU) y sensores	**1 – 10 ms**	**Miliwatts (mW) o $\m**

3.2 ¿Por qué TinyML en Ingeniería Biomédica?

El procesamiento de IA directamente en el sensor o microcontrolador (*on-device AI*) aporta cuatro ventajas determinantes:

1. **Latencia Determinista en Tiempo Real (1–10 ms):** No depende de retardos de red celular o Wi-Fi para emitir un veredicto crítico (por ejemplo, parada cardíaca en desfibrilador o desconexión de bomba).
2. **Privacidad y Cumplimiento Normativo (Habeas Data / HIPAA):** Las imágenes y las bioseñales del paciente se procesan localmente en la memoria SRAM del chip y se descartan inmediatamente; **los datos sensibles nunca abandonan el dispositivo**.
3. **Autonomía y Confiabilidad sin Red:** El sistema sigue operando en ambulancias, zonas rurales o ante caídas del servidor hospitalario.
4. **Eficiencia Energética Extrema:** Transmitir 1 byte por radiofrecuencia (Wi-Fi/4G) gasta hasta **1.000 veces más energía** que computar ese mismo byte localmente en una MCU.

---

4. Práctica de Laboratorio: Sistemas Inteligentes en Informática Médica

> **!** Información del Laboratorio
>
> - **Docente:** Prof. Jefferson Sarmiento Rojas, Ph.D (c)
> - **Plataforma de Hardware:** Kit Seeed Studio XIAO ESP32S3 Sense
> - **Entorno de Programación:** Arduino IDE 2.x con biblioteca ESP32 de Espressif Systems
> - **Herramienta TinyML:** SenseCraft AI Studio (Web Toolkit)
> - **Protocolo de Integración:** MQTT (Message Queuing Telemetry Transport)

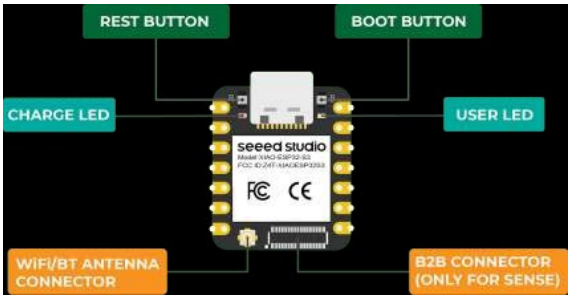
4.1 Características de la Plataforma XIAO ESP32S3 Sense

La **XIAO ESP32S3 Sense** es una plataforma de cómputo embebido de alto rendimiento diseñada específicamente para aplicaciones de visión artificial y reconocimiento de audio mediante TinyML.

- **Procesador:** MCU Espressif ESP32-S3 de 32 bits, doble núcleo Xtensa LX7 a 240 MHz, con soporte de instrucciones vectoriales para aceleración de redes neuronales.
- **Módulo de Visión:** Cámara desmontable **OV2640** (hasta 1600 × 1200 px), intercambiable con sensor OV5640.
- **Módulo de Audio:** Micrófono digital PDM integrado en la placa base con interfaz I2S.



(a) Tarjeta Seeed Studio XIAO ESP32S3 Sense con cámara OV2640 acoplada



(a) Disposición general de pines (Pinout) de la XIAO ESP32S3 Sense

- **Memoria Expandida: 8 MB de PSRAM** (Pseudo-Static RAM vía interfaz OPI) y **8 MB de memoria Flash**, esenciales para albergar tensores neuronales. Ranura para tarjeta microSD FAT32 de hasta 32 GB.
- **Comunicaciones Inalámbricas:** Wi-Fi 802.11 b/g/n (2.4 GHz) y Bluetooth Low Energy (BLE 5.0) con conector para antena externa U.FL.
- **Factor de Forma:** Dimensiones ultra compactas de **21 × 17.5 mm**.
- **Gestión de Energía:** Circuito de carga para batería LiPo con consumo en sueño profundo (*Deep Sleep*) de apenas **14 μA**.

4.2 Planteamiento del Problema Hospitalario

Durante las rondas de inspección y vigilancia técnica hospitalaria, los practicantes de Ingeniería Biomédica se enfrentan a una amplia diversidad de equipos médicos (bombas de infusión, monitores de signos vitales, desfibriladores, analizadores de ECG) de distintas marcas y generaciones.

La falta de familiaridad inmediata con un dispositivo específico puede derivar en registros erróneos en el sistema de gestión hospitalaria o demoras para consultar el **manual técnico del fabricante**, poniendo en riesgo la calidad asistencial.

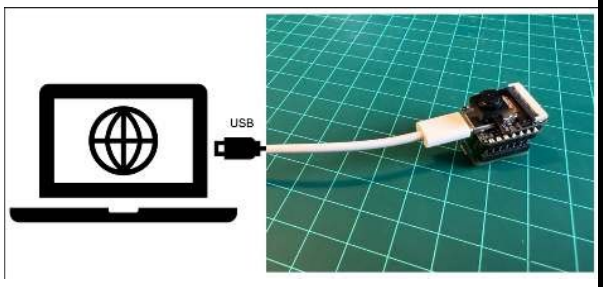
Reto de Ingeniería: Desarrollar un sistema embebido inteligente sobre la placa **XIAO ESP32S3 Sense** que capture imágenes del entorno, identifique el equipo médico en tiempo real mediante un modelo TinyML optimizado y transmita vía **MQTT** el identificador del equipo junto con el enlace URL a su manual de operación, facilitando el acceso instantáneo desde el smartphone o tablet del practicante.

4.3 Ejercicio 1: Configuración del Entorno de Desarrollo (Arduino IDE)

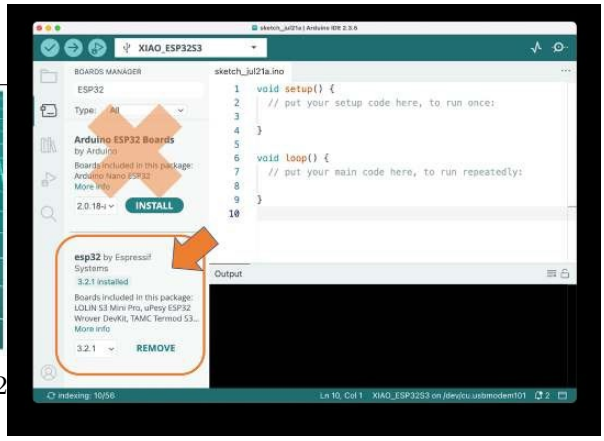
1. Conecte la tarjeta XIAO ESP32S3 Sense al computador mediante un cable USB-C de datos.
2. Abra Arduino IDE (versión 2.x recomendada).
3. Diríjase a **Herramientas** → **Administrador de placas** (o el ícono lateral de placas).
4. En el buscador escriba `ESP32` e instale el paquete oficial **`esp32` by Espressif Systems**.

> ⚠ Precaución con la Selección de Paquete
>
> No instale el paquete *"Placas Arduino ESP32 de Arduino"*, ya que está destinado a la placa Arduino Nano ESP32 y generará errores de compilación con las bibliotecas de Seeed Studio.

5. En el menú superior de selección de placa, escriba `xiao esp32s3` y seleccione la placa **`XIAO_ESP32S3`** junto con el puerto COM asignado por el sistema operativo.



(a) Administrador de Placas: Instalación de esp32 by Espressif



(a) Selección de placa XIAO_ESP32S3 y puerto COM

4.4 Ejercicio 2: Verificación de Hardware (Blink y Monitor Serial)

Este ejercicio confirma que la placa responde a la programación y que el enlace de comunicación serie está operativo.

```
// Ejercicio 2: Verificación de LED y Telemetría Serial
// Placa: Seeed Studio XIAO ESP32S3 Sense

#define LED_BUILT 21  // Pin asignado al LED integrado en placa

void setup() {
  pinMode(LED_BUILT, OUTPUT); // Configura pin como salida digital
  Serial.begin(9600);         // Inicializa comunicación serial a 9600 bps
}

void loop() {
  digitalWrite(LED_BUILT, LOW);   // Enciende el LED (lógica invertida)
  Serial.println("LED Encendido");
  delay(1000);                    // Pausa de 1 segundo

  digitalWrite(LED_BUILT, HIGH);  // Apaga el LED
  Serial.println("LED Apagado");
  delay(1000);                    // Pausa de 1 segundo
}
```

Abra el **Monitor Serie** en Arduino IDE configurado a `9600 baudios` y compruebe la alternancia de estados.

4.5 Ejercicio 3: Adquisición de Señal Acústica con Micrófono PDM e I2S

La XIAO ESP32S3 Sense integra un micrófono digital **PDM (Pulse Density Modulation)** que entrega muestras de audio mediante el bus de comunicación **I2S**. Este periférico resulta fundamental para proyectos de fonocardiografía digital o detección acústica de anomalías en turbinas y motores médicos.

```
// Ejercicio 3: Lectura en Tiempo Real del Micrófono PDM Integrado
// Placa: Seeed Studio XIAO ESP32S3 Sense

#include <ESP_I2S.h>

I2SClass I2S;

void setup() {
  Serial.begin(115200);
```

```
  while (!Serial) {
    // Espera hasta que el monitor serial esté conectado
  }

  // Configura pines del micrófono PDM: Pin 42 (Reloj PDM) y Pin 41 (Datos PDM)
  I2S.setPinsPdmRx(42, 41);

  // Inicializa I2S en modo PDM RX a 16 kHz, 16 bits por muestra, canal Mono
  if (!I2S.begin(I2S_MODE_PDM_RX, 16000, I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_MONO)) {
    Serial.println("¡Error al inicializar el bus I2S del micrófono!");
    while (1); // Detención por error
  }
  Serial.println("Micrófono PDM inicializado con éxito.");
}

void loop() {
  // Lectura de una muestra de audio digital
  int sample = I2S.read();

  // Filtrado de valores espurios o silencios triviales
  if (sample != 0 && sample != -1 && sample != 1) {
    Serial.println(sample); // Imprime el valor instantáneo de amplitud
  }
}
```

Visualización de la Bioseñal Acústica

Abra **Herramientas** → **Serial Plotter** en Arduino IDE para observar la forma de onda acústica en tiempo real mientras habla o genera sonidos cerca de la placa.

4.6 Ejercicio 4: Grabación de Audio WAV en MicroSD y Habilitación de OPI PSRAM

Dado que un buffer de audio de varios segundos satura los pocos cientos de kilobytes de memoria SRAM interna del microcontrolador, es obligatorio habilitar la memoria externa **PSRAM de 8 MB** integrada en el encapsulado del XIAO.

Activación de PSRAM en Arduino IDE:

Vaya al menú: **Herramientas → PSRAM → "OPI PSRAM"**.

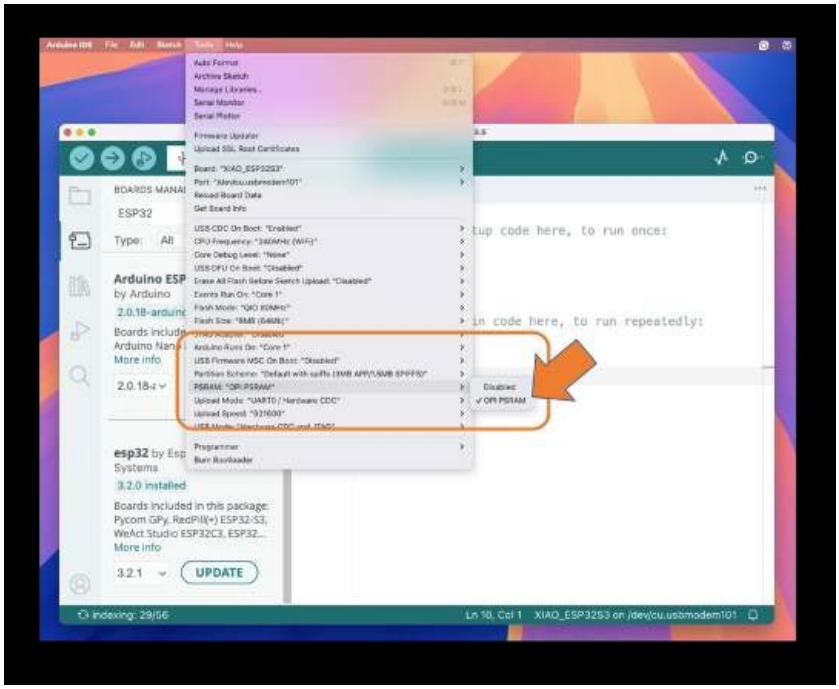


Figure 10: Habilitación de OPI PSRAM en el menú de herramientas de Arduino IDE

```
// Ejercicio 4: Grabador de Audio WAV en Tarjeta MicroSD
// Placa: Seeed Studio XIAO ESP32S3 Sense (Requiere OPI PSRAM habilitada)

#include "ESP_I2S.h"
#include "FS.h"
#include "SD.h"

void setup() {
  I2SClass i2s;
  uint8_t *wav_buffer;
  size_t wav_size;

  Serial.begin(115200);
  while (!Serial) delay(10);

  Serial.println("Iniciando bus I2S para micrófono...");
  i2s.setPinsPdmRx(42, 41); // Pines PDM Clock y PDM Data

  if (!i2s.begin(I2S_MODE_PDM_RX, 16000, I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_MONO)) {
```

```
    Serial.println("Error en inicialización de I2S");
    while (1);
  }

  Serial.println("Inicializando ranura MicroSD...");
  // Pin CS de la ranura microSD en la XIAO ESP32S3 Sense: Pin 21
  if (!SD.begin(21)) {
    Serial.println("¡Error al montar la tarjeta MicroSD! Verifique formato FAT32.");
    while (1);
  }
  Serial.println("MicroSD montada correctamente.");

  Serial.println("Grabando 20 segundos de audio del entorno...");
  // Graba 20 segundos alojando el buffer en la PSRAM
  wav_buffer = i2s.recordWAV(20, &wav_size);

  File file = SD.open("/audio_biomedico.wav", FILE_WRITE);
  if (!file) {
    Serial.println("Error al abrir archivo para escritura en SD.");
    return;
  }

  Serial.println("Escribiendo datos binarios WAV en SD...");
  if (file.write(wav_buffer, wav_size) != wav_size) {
    Serial.println("Error en la escritura del buffer completo.");
    return;
  }
  file.close();

  Serial.println("Grabación finalizada con éxito: /audio_biomedico.wav");
}

void loop() {
  delay(1000);
  Serial.print(".");
}
```

4.7 Ejercicio 5 (Resolución del Reto): Visión Artificial Embebida con SenseCraft AI Studio

En este ejercicio aplicaremos el flujo de **Edge AI** sin escribir código a bajo nivel, utilizando la plataforma web oficial **SenseCraft AI Studio** de Seeed Studio.

```
[ Navegador Web ]  (Web Serial API)  [ XIAO ESP32S3 Sense ]   [ Cámara OV2640 ]
```

Paso 1: Conexión y Reconocimiento de la Placa

1. Abra Google Chrome o Microsoft Edge y acceda a SenseCraft Web Toolkit.
2. En la lista de dispositivos compatibles, seleccione **XIAO ESP32S3 Sense**.
3. Haga clic en **Connect Device**, seleccione el puerto COM correspondiente y confirme la conexión.

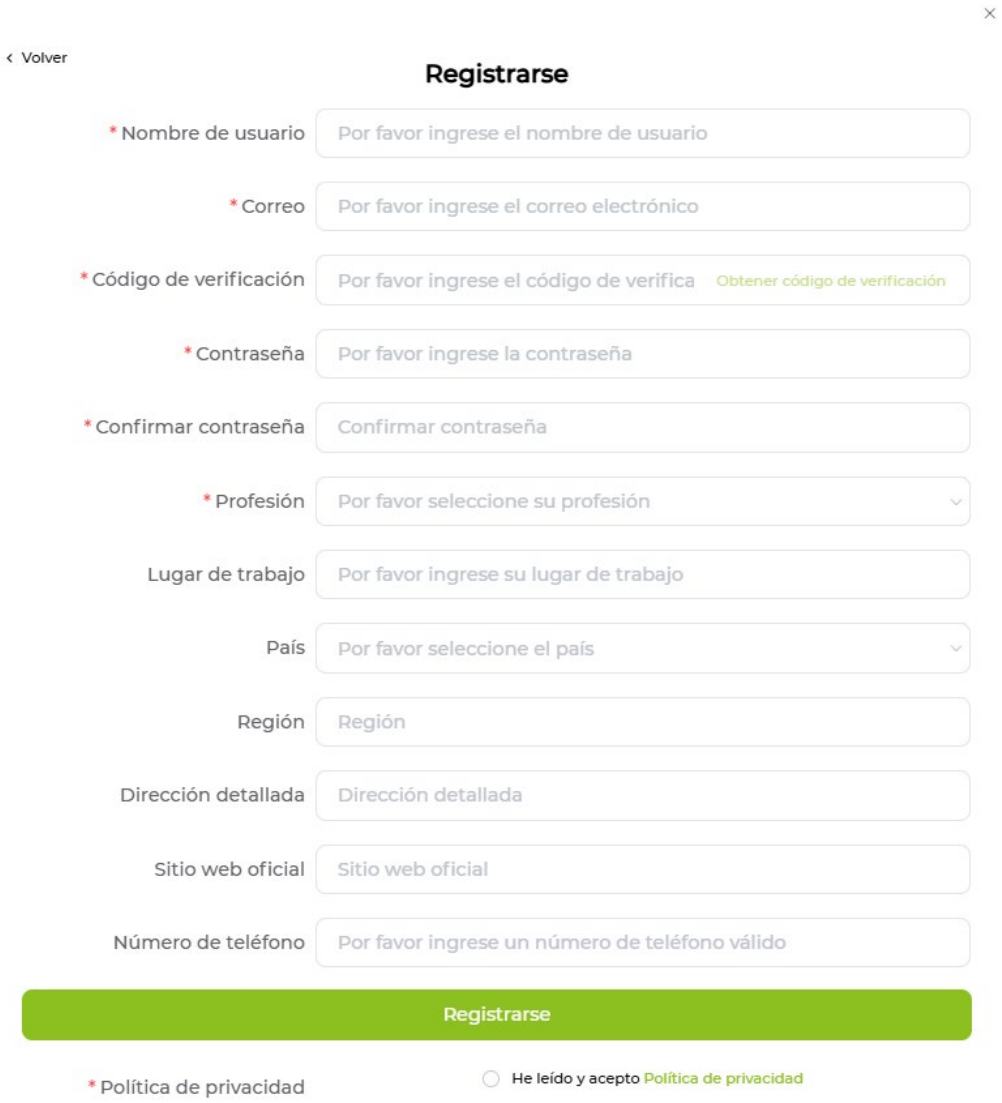


Figure 11: Selección de la tarjeta XIAO ESP32S3 Sense en SenseCraft AI Studio

---

Paso 2: Verificación del Streaming de Cámara

Cargue el firmware de prueba para verificar que el sensor OV2640 enfoca adecuadamente y transmite vídeo a la ventana del navegador.

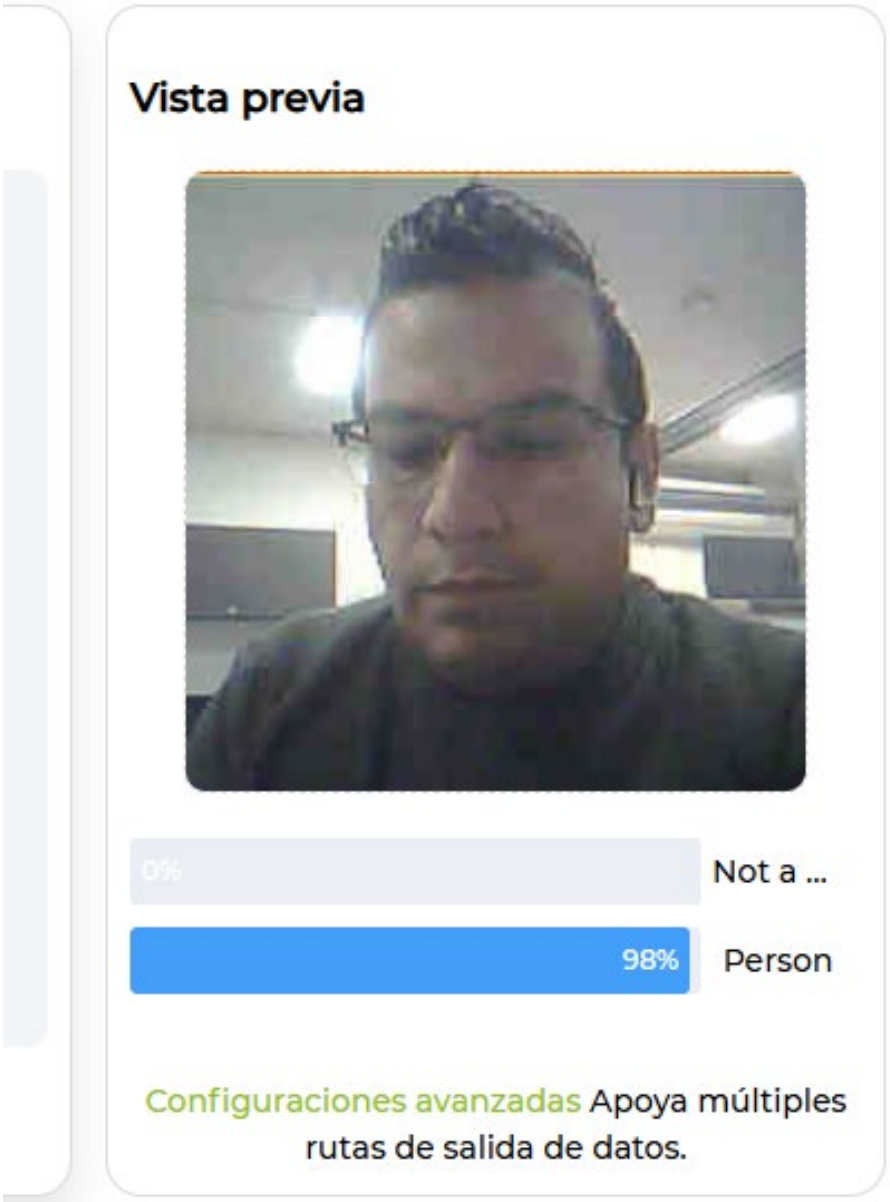


Figure 12: Verificación de funcionamiento y captura de vídeo con cámara OV2640

Paso 3: Construcción del Dataset Biomédico

Para resolver el reto hospitalario, defina al menos **3 o 4 clases de equipos médicos**:

- **Clase 1:** `Bomba_Infusion`
- **Clase 2:** `Monitor_Signos_Vitales`
- **Clase 3:** `Desfibrilador`
- **Clase 4 (Opcional):** `Electrocardiografo` o `Fondo_Vacio` (para evitar falsos positivos).

Mantenga presionado el botón de captura en cada clase mientras rota el dispositivo médico frente al lente de la cámara, variando la distancia, la inclinación y la iluminación (mínimo **50 a 100 imágenes por clase**).

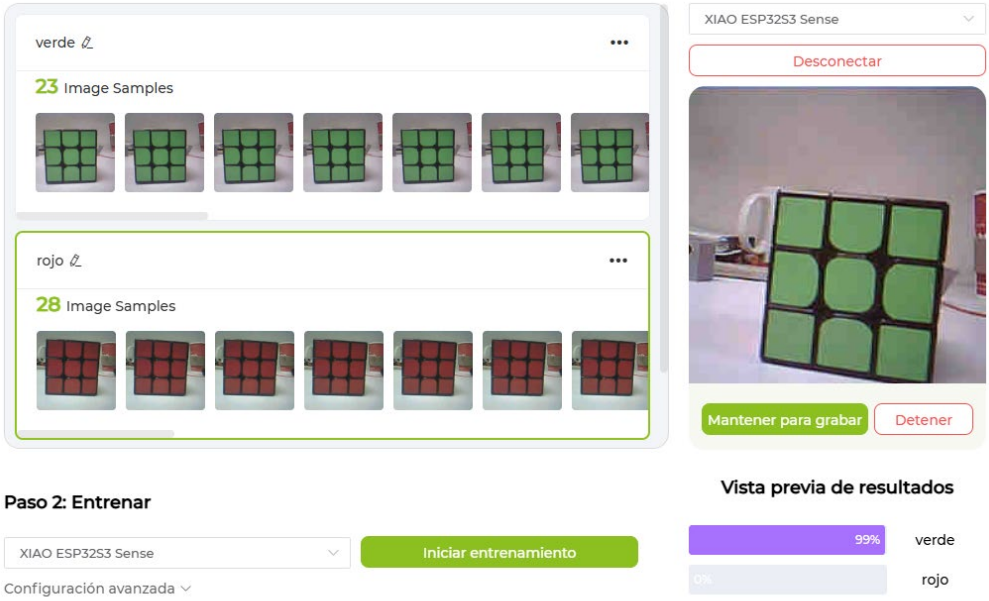


Figure 13: Creación del dataset: Captura de muestras visuales para cada clase de dispositivo

---

Paso 4: Entrenamiento y Optimización del Modelo

1. Haga clic en **Start Training**.
2. SenseCraft procesará el dataset mediante una arquitectura convolucional ligera optimizada para microcontroladores (MobileNetV2 / SqueezeNet con cuantización INT8).
3. Revise la matriz de confusión y el valor de precisión (*Accuracy*). Si alguna clase muestra confusión, capture más imágenes con diferentes ángulos de luz.



Figure 14: Entrenamiento del clasificador neuronal embebido en SenseCraft

---

Paso 5: Despliegue en la Tarjeta (Flashing)

Haga clic en **Deploy Model**. La plataforma compilará el modelo cuantizado en formato TensorFlow Lite for Microcontrollers (TFLite Micro) y lo grabará directamente en la memoria Flash y PSRAM de la XIAO ESP32S3 Sense.

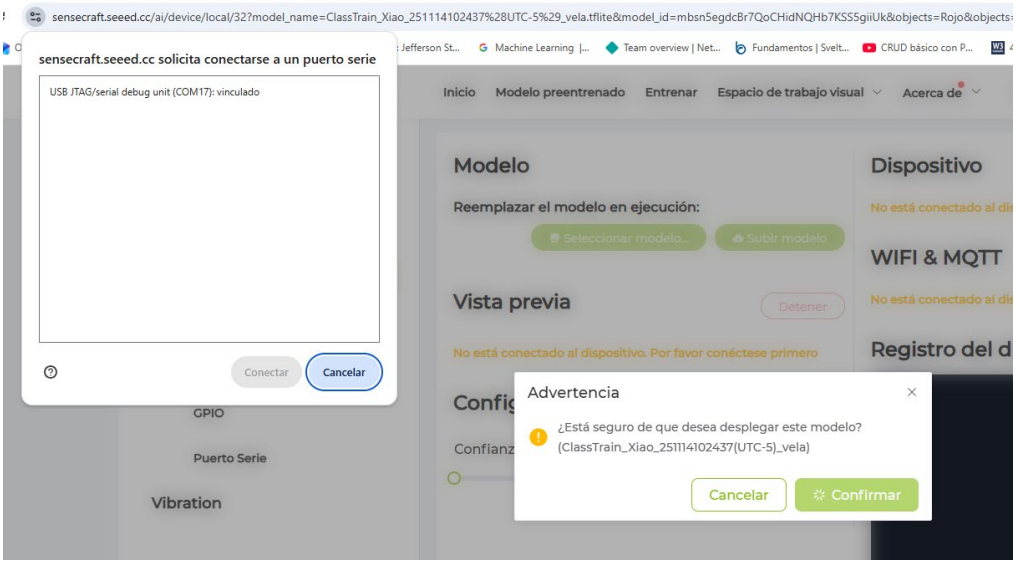


Figure 15: Flasheo y despliegue del modelo neuronal compilado en la MCU

Paso 6: Verificación de Inferencia en Tiempo Real

Una vez finalizada la carga, la tarjeta ejecutará el bucle de inferencia de manera totalmente autónoma a una velocidad de **15 a 30 FPS**. Al apuntar la cámara a un monitor de signos vitales o a una bomba de infusión, la etiqueta y el porcentaje de confianza se visualizarán en pantalla.

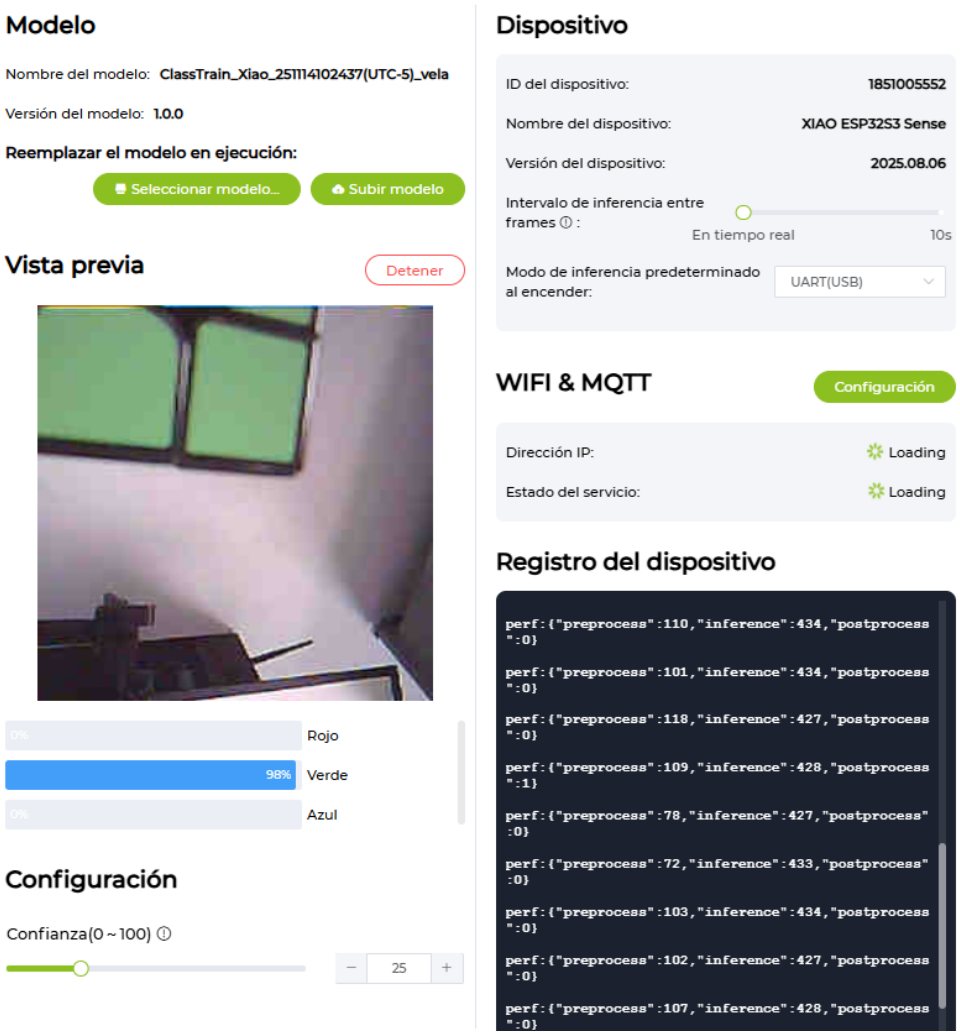


Figure 16: Validación de inferencia local en tiempo real con alta certeza predictiva

---

4.8 Integración con Arquitectura IoT / MQTT (Solución Integral)

Para completar el objetivo del reto hospitalario, el sistema embebido debe enlazar la inferencia local con la red IoT hospitalaria vista en la Clase 12.

Cuando el modelo detecte una clase con confianza superior al **85%**, el microcontrolador publica un mensaje JSON estructurado al broker MQTT hospitalario en el tópico:

`hospital/piso3/uci/equipos/identificacion`

```
{
  "timestamp": "2026-10-14T09:30:00Z",
  "dispositivo_sensor": "XIAO-ESP32S3-CAM-01",
  "equipo_detectado": "Bomba de Infusión Volumétrica",
  "modelo": "B. Braun Infusomat Space",
  "confianza": 0.94,
  "manual_tecnico_url": "https://hospital.urosario.edu.co/manuales/bbraun_infusomat.pdf",
  "estado_mantenimiento": "Vigente - Calibración 2026-II"
}
```


De este modo, cualquier smartphone o tablet del servicio hospitalario suscrito al tópico recibe de inmediato el enlace al manual operativo sin requerir búsqueda manual en archivadores físicos.

Bibliografía y Referencias

1. **Reddi, V. J., et al.** (2024). *Introduction to Machine Learning Systems: From Logic to Arithmetic (Neural Computation)*. Harvard University / MLSysBook. Disponible en: https://mlsysbook.ai/vol1/nn_computation/nn_computation.html
2. **Sarmiento Rojas, J.** (2026). *Actividad Práctica de Laboratorio: Sistemas Inteligentes en Informática Médica*. Programa de Ingeniería Biomédica, Escuela de Medicina y Ciencias de la Salud, Universidad del Rosario. Descargar PDF
3. **Warden, P., & Situnayake, D.** (2019). *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media.
4. **Seeed Studio.** (2024). *Getting Started with Seeed Studio XIAO ESP32S3 (Sense)*. Seeed Wiki Documentation. Disponible en: https://wiki.seeedstudio.com/xiao_esp32s3_getting_started/
5. **Espressif Systems.** (2024). *ESP-IDF Programming Guide: ESP32-S3 Technical Reference Manual (Vector Instructions & I2S PDM)*. Disponible en: https://docs.espressif.com/